

KUNG FOOD

Rapport de projet — Phases 1 et 2

Projet Creative-Yumland

Membres	Bilel — Liam — Riyane
Objet	Phase 1 + Phase 2

Phase 1

La phase 1 portait sur la création de l'interface graphique côté client, avec un affichage statique en HTML et CSS.

Pages réalisées

- **Accueil (Acceuil.html)** : présentation du site, horaires et plats les plus commandés.
- **Menu (Menu.html)** : affichage des plats avec barre de recherche et filtres.
- **Inscription (Inscription.html)** : création de compte.
- **Connexion (page de connexion.html)** : connexion au profil utilisateur.
- **Profil (page de profil.html)** : informations utilisateur et historique des commandes.
- **Administrateur (Admin.html)** : liste des utilisateurs et accès aux profils.
- **Commandes (commandes.html)** : interface restaurateur pour le suivi des commandes.
- **Livraison (page de livraison.html)** : interface livreur avec les informations client. -
- **Notation (Note.html)** : notation du service et des plats.

Phase 2

La phase 2 a consisté à transformer la maquette statique en une application PHP dynamique utilisant des fichiers JSON pour stocker les données côté serveur. Les pages principales ont été renommées en PHP et le projet a été réorganisé afin de séparer les vues, les scripts réutilisables et les données.

Répartition des tâches

Membre	Travaux principaux
Bilel	Utilisateurs, inscription, connexion, page administrateur, accès aux profils, affichage des actions de gestion de compte, préparation de la partie paiement.

Liam	Options et commandes, détail des commandes, attribution des livreurs, historique client, panier, validation de commande, suivi des statuts, préparation immédiate ou différée.
------	--

Riyane		Menus et plats, conversion HTML vers PHP, rangement de l'arborescence PHP, séparation des fichiers de données JSON, création de la page panier, dynamisation de la page de livraison, terminer la filtration du menu, dynamiser le panier avec l'ajout de la fonctionnalité de l'ajout d'article et le retrait de produit.

Fonctionnalités mises en place

- Inscription fonctionnelle avec ajout d'un nouvel utilisateur dans utilisateurs.json. - Connexion fonctionnelle avec redirection selon le rôle et mise à jour de la dernière connexion. - Affichage de la liste des utilisateurs côté administrateur, consultation des profils et modification des champs de gestion (rôle, statut, blocage).
- Lecture dynamique des plats et des menus depuis plats.json.
- Ajout et suppression d'articles dans le panier.
- Validation d'une commande avec enregistrement dans commandes.json.
- Gestion d'une commande immédiate ou planifiée pour plus tard.
- Affichage, filtrage et mise à jour des statuts des commandes côté restaurateur. - Attribution d'un livreur à une commande et affichage des informations de livraison. - Affichage de l'historique des commandes côté client et présence d'une page de notation liée au parcours de commande.

Format des fichiers JSON

Dans ce projet, nous avons choisi d'utiliser des fichiers JSON pour stocker les données au lieu d'une base de données. Ce choix permet d'avoir une structure simple, lisible et facile à manipuler en PHP. Les données sont organisées dans trois fichiers principaux : utilisateurs.json, plats.json et commandes.json.

Le fichier utilisateurs.json contient la liste des comptes du site. Chaque utilisateur est stocké sous forme d'un objet avec des champs comme id, Nom, Prenom, Mail, Mdp, age, numero, adresse, interphone, rôle, date_inscription, derniere_connexion, statut, bloquer et Commandes. Dans le code, le champ age correspond en réalité à une date de naissance, et non à un âge numérique. Le champ rôle permet de distinguer les différents types d'utilisateurs comme client, administrateur, restaurateur ou livreur. Pour certains livreurs, on trouve aussi le champ disponible_livraison.

Le fichier plats.json contient l'ensemble des produits affichés sur le site. Chaque élément possède généralement les champs nom, description, prix, catégorie et filters. Certains plats possèdent aussi un champ image. Pour les menus, on trouve en plus un champ Composition qui contient la liste des éléments inclus dans le menu. Cela signifie donc que tous les objets n'ont pas exactement la même structure, ce qui correspond bien à ce qui est réellement codé.

Le fichier commandes.json contient les commandes passées par les clients. Chaque commande est enregistrée avec des champs comme id, client_id, client_nom, articles, nombre_articles, total, type_service, moment_preparation, date_retrait_livraison, adresse_livraison, commentaire, statut, paiement_statut et date_creation. Le champ articles est lui-même un tableau contenant les produits commandés, avec pour chacun nom, prix_unitaire, quantite et sous_total. Lorsqu'une commande est notée après livraison, un champ note_commande peut aussi être ajouté avec les notes et le commentaire du client.

Le format choisi est donc un ensemble de tableaux JSON contenant des objets structurés selon le type de donnée stockée. Ce choix correspond au fonctionnement réel du projet, car les fichiers sont lus et modifiés directement par les pages PHP pour gérer les utilisateurs, l'affichage des plats et le suivi des commandes.

Problèmes rencontrés

Pendant le projet, notre groupe a rencontré plusieurs difficultés techniques. D'abord, nous avons eu du mal à bien concevoir le format des données, car il fallait organiser correctement les utilisateurs, les produits, les commandes et les liens entre toutes ces informations. Ensuite, la gestion des différents rôles utilisateurs a aussi posé problème, car chaque profil (client, restaurateur, livreur, administrateur) devait avoir des droits et des fonctionnalités différentes.

Nous avons également rencontré des difficultés pour gérer le panier et les commandes planifiées, puisqu'il fallait prendre en compte à la fois les commandes immédiates et celles prévues pour plus tard. L'intégration du paiement avec l'API CYBank a aussi été compliquée, car nous devions comprendre le fonctionnement d'un service externe et relier correctement le paiement à la commande. Enfin, la création de pages dynamiques côté serveur en PHP a demandé plus de travail que prévu, notamment pour afficher les bonnes informations selon l'utilisateur connecté et l'état des commandes.

Dans l'ensemble, ça été sûrement l'étape la plus dur du projet car on a dû s'adapter aux différents codes des membres du groupe mais cela est aussi bénéfique que on a du comprendre tout dans sa globalité et cela sera forcément bénéfique pour la finalisation du projet

Phase 3

La phase 3 a consisté à ajouter du code JavaScript côté client afin de rendre les pages dynamiques, et à utiliser des requêtes asynchrones pour modifier le contenu des pages sans rechargement complet.

Répartition des tâches

Membre	Travaux principaux
Bilel	Validation des formulaires côté client (inscription, connexion) en JavaScript : vérification du contenu de chaque champ (email, mot de passe, numéro de téléphone, date, etc.) avec affichage de messages d'erreur sans recharger la page. La requête HTTP ne part que si l'ensemble du formulaire est valide. Blocage/déblocage d'un utilisateur côté administrateur en utilisant des requêtes asynchrones obligatoirement. Si un utilisateur est bloqué, sa session courante est terminée sur-le-champ. Si une commande a déjà été payée par cet utilisateur, son cycle est terminé malgré tout. Indication par le livreur qu'une livraison qui lui a été assignée vient d'être effectuée. Notation d'une commande livrée par le client (une seule fois par commande).
Liam	Affichage d'un compteur de caractères en temps réel pour les champs limités en taille (email, mot de passe). Ajout et suppression de produits sur une commande déjà payée mais pas encore en préparation : le montant total se met à jour automatiquement. Si la commande est plus chère, le client effectue un paiement complémentaire de la différence. Si la commande est moins chère, le client obtient un ticket de réduction à utiliser ultérieurement. Gestion des statuts des commandes côté restaurateur : passage de l'état payée à en préparation, puis à l'état prête, avec assignation d'un livreur disponible.

Riyane	Changement de charte graphique (mode clair/sombre) via bouton, avec sauvegarde du choix dans un cookie et chargement du nouveau fichier CSS sans recharger la page. Filtres et tris asynchrones sur la page de présentation des produits : les filtres récupèrent uniquement les données correspondantes via une connexion asynchrone, les tris sont effectués sur les données déjà chargées. Modification des informations du profil utilisateur transmises au serveur en connexion asynchrone obligatoirement.
--------	--

Fonctionnalités mises en place

- Changement de charte graphique (mode clair/sombre) via bouton, avec mémorisation dans un cookie
- Validation côté client de tous les formulaires (inscription, connexion) avec messages d'erreur en JavaScript, sans rechargement de page.
- Compteur de caractères restants en temps réel sur les champs limités en taille.
- Affichage et masquage du mot de passe via une icône œil.
- Modification des informations du profil utilisateur via requête asynchrone.
- Filtres asynchrones et tris côté client sur la page des produits.
- Modification d'une commande déjà payée (ajout/suppression d'articles) avec mise à jour du total et paiement complémentaire si nécessaire.
- Changement de statut des commandes côté restaurateur et assignation d'un livreur.
- Blocage/déblocage asynchrone d'un utilisateur côté administrateur avec fin de session immédiate.
- Indication de livraison effectuée par le livreur.
- Notation d'une commande livrée par le client (une seule fois).

Problèmes rencontrés

Durant la phase 3, plusieurs difficultés ont été rencontrées. Tout d'abord, des corrections ont été apportées au site suite aux remarques formulées lors de la phase 2. Sur le plan technique, le fichier variables.css ainsi que l'ensemble des fichiers CSS ont dû être largement retravaillés, car certains éléments textuels ne s'adaptaient pas correctement au mode clair ou sombre du site. La compréhension et la mise en œuvre du fetch pour les requêtes asynchrones a également représenté une difficulté majeure, nécessitant un temps d'adaptation important. Par ailleurs, des modifications ont dû être apportées au code HTML afin d'y intégrer de nouvelles classes et identifiants rendus nécessaires par les nouvelles fonctionnalités JavaScript. Enfin, l'intégration du code dans une version antérieure du projet a complexifié le travail, obligeant à adapter les développements réalisés au contexte existant.